

Bases de datos y patrones de acceso a datos

GameShelf · Modelado · SQL · Repository · Unit of Work · CRUD vs CQRS

Esta sesión es el puente entre los datos y el código. No vamos a programar todavía — vamos a pensar en cómo se organiza la información antes de escribir una sola línea.

El hilo conductor es GameShelf: una app donde los usuarios registran los videojuegos que han jugado y les ponen una nota del 1 al 10.

1 Modelado relacional

Antes de crear una base de datos hay que identificar las entidades del dominio y cómo se relacionan entre ellas.

? *¿Qué información necesitamos guardar en GameShelf? ¿Qué entidades identificas?*



Piensa en los "sustantivos" del sistema: ¿qué cosas existen? ¿qué sabe cada una de sí misma?

Una vez tengamos las entidades, hay que pensar en cómo se relacionan.

? *Un juego puede tener varios géneros y un género puede agrupar muchos juegos. ¿Cómo representamos eso en tablas?*



¿Basta con una columna en Game? ¿O necesitamos algo más entre las dos tablas?

? *Si guardamos el género como texto directamente en cada fila de Game, ¿qué problemas puede causar cuando hay miles de juegos?*



Piensa en consistencia, repetición y qué pasa si alguien escribe 'rpg' en minúsculas y otro 'RPG'.

El esquema que vamos a construir juntos tiene esta estructura. Complétalo mientras lo discutimos:

```
// ¿Qué columnas tiene Genre?
Genre
  id    (PK)
  ???

// ¿Qué columnas tiene Game? ¿Qué FK necesita?
Game
  id    (PK)
  ???

// ¿Qué hace esta tabla? ¿Qué PK tiene?
GameGenre
  ???
```

```
// ¿Qué relación tiene Review con Game?
Review
  id    (PK)
  ???
```

2 Consultas SQL

Tenemos el esquema. Ahora lo usamos. Cada consulta que vamos a ver responde a una pregunta de negocio real.

Consulta 1 — ¿Cuántas reviews tiene cada juego?

Necesitamos contar filas agrupadas por juego. Queremos que aparezcan también los juegos sin ninguna review.

? ¿Qué diferencia hay entre JOIN y LEFT JOIN? ¿Cuál usarías aquí y por qué?

💡 ¿Qué pasa con los juegos que no tienen reviews si usas un JOIN normal?

Consulta 2 — Nota media por juego (solo los que superan un mínimo)

Queremos calcular la media de scores y filtrar los grupos que no llegan a 7.

? ¿Cuál es la diferencia entre WHERE y HAVING? ¿En qué momento del proceso actúa cada uno?

💡 WHERE filtra filas antes de agrupar. HAVING filtra grupos ya formados. ¿Cuál necesitas para filtrar por la media?

Consulta 3 — ¿Qué géneros tiene cada juego?

Para responder esto tenemos que pasar por la tabla pivot GameGenre.

? ¿Por qué necesitamos dos JOINS para llegar desde Game hasta Genre? ¿Qué papel juega GameGenre?

Consulta 4 — Nota media de juegos RPG

Combinamos todo lo anterior: filtro por género, JOIN de tres tablas, GROUP BY y media calculada.

? ¿El WHERE filtra antes o después de calcular la media? ¿Qué implicaciones tiene?

3 Repository Pattern

Ya sabemos hacer las consultas. La pregunta ahora es: ¿dónde vive ese SQL dentro de la aplicación?

? *¿Qué problemas surgen si ponemos las consultas directamente dentro del controlador que responde peticiones HTTP?*

💡 Piensa en qué pasa si el esquema cambia, en cómo lo testearías, y en cuántas responsabilidades tiene ese controlador.

Repository Pattern

Una clase cuya única responsabilidad es hablar con la base de datos. El resto del sistema le habla a ella, no al SQL.

? *El controlador recibe un `IGameRepository`, no un `GameRepository`. ¿Por qué una interfaz y no la clase directamente?*

💡 ¿Qué pasa en los tests si dependes de la clase concreta? ¿Puedes sustituirla fácilmente?

4 Unit of Work

Cuando añadimos un juego nuevo, necesitamos insertar en `Game` y también en `GameGenre` (una fila por cada género).

? *¿Qué pasa si el `INSERT` en `GameGenre` falla después de que el de `Game` ya se confirmó? ¿En qué estado quedan los datos?*

💡 ¿Conoces el concepto de transacción? ¿Qué garantiza?

Unit of Work

Agrupar varias operaciones de escritura en una sola transacción. O se confirman todas, o no se confirma ninguna.

? *En EF Core, ¿en qué momento se ejecuta realmente el `INSERT` en la base de datos? ¿Qué hace `SaveChangesAsync()`?*

5 CRUD vs CQRS

Dos formas distintas de organizar el acceso a datos. No son enemigos — son herramientas para contextos diferentes.

CRUD — Create, Read, Update, Delete

Una sola vía de acceso. El mismo modelo sirve para leer y para escribir. Simple de implementar y mantener.

CQRS — Command Query Responsibility Segregation

Separa las operaciones de escritura (Commands) de las de lectura (Queries). Cada lado puede tener su propio modelo optimizado.

? *¿En qué situación empezaría a ser un problema usar el mismo modelo para leer y para escribir?*



Imagina que tienes una consulta que necesita JOINS de 5 tablas y 3 agregaciones, pero escribir es un simple INSERT. ¿Qué compromiso estás haciendo al usar el mismo modelo para ambas cosas?

? *En GameShelf usamos CRUD. ¿En qué momento o con qué funcionalidad nueva tendría sentido plantearse CQRS?*